

# Programmazione e calcolo scientifico

## Esercitazione 5 - Algoritmi di ordinamento, parte 2

21 aprile 2026

**NOTA:** si ricorda che copiare/incollare testo dai file PDF ad un editor di testo/codice potrebbe non preservare la formattazione, producendo risultati inattesi.

### 1 Algoritmi di ordinamento, parte 2

Si estenda quanto fatto per la precedente esercitazione 4 con gli algoritmi Mergesort e Quicksort visti a lezione e riportati in pseudocodice nel seguito.

Si faccia attenzione al fatto che frequentemente nello pseudocodice gli indici dei vettori iniziano da uno e non da zero. Si faccia pertanto attenzione a non commettere errori di off-by-one nell'implementazione degli algoritmi.

Una volta implementati gli algoritmi, si aggiungano i test unitari che, chiamando la funzione `is_sorted()` precedentemente scritta, verifichino l'effettivo funzionamento dell'algoritmo. Si ricordano le richieste dell'esercitazione precedente rispetto ai test:

- Testare gli algoritmi di ordinamento su 100 vettori di dimensioni scelte a caso
- Per ognuna delle dimensioni si testi gli algoritmi su vettori riempiti in modo casuale e che includano numeri sia negativi sia positivi.
- Si crei un vettore di `std::string` e lo si riempia con una decina di stringhe scelte a piacere, si verifichi quindi che gli algoritmi ordinano correttamente anche le stringhe.

Si confrontino i tempi di Bubblesort, Selectionsort ed Insertionsort con i tempi di Quicksort e di Mergesort su vettori piccoli (indicativamente, dimensioni inferiori a 100) e si determini se esiste una dimensione al disotto della quale gli algoritmi quadratici sono più veloci degli algoritmi logaritmici. Se sì, si proponga una versione modificata di Quicksort che, al disotto di tale soglia, utilizzi l'algoritmo quadratico più veloce per fare l'ordinamento.

Si confrontino infine i tempi delle proprie implementazioni con i tempi misurati per `std::sort()` e si scriva un file di testo di qualche riga contenente le proprie osservazioni.

Si ricordi che, se si vuole ordinare il vettore `v` utilizzando `std::sort()`, questa va chiamata nel modo seguente:

```
std::sort( v.begin(), v.end() );
```

**Attenzione:** in questa esercitazione è necessario misurare in modo relativamente preciso il tempo di esecuzione degli algoritmi. A tal fine si proceda come segue:

- Si preallochino un centinaio di vettori della dimensione che si vuole testare (i vettori possono essere a loro volta memorizzati in un vettore, quindi in un `std::vector<std::vector<T>>` e si inizializzino con numeri casuali
- Tramite un ciclo `for`, si ordinino tutti i vettori, misurando il tempo totale di esecuzione con delle `tic()/toc()` all'esterno del ciclo `for`
- Si calcoli la media, dividendo il tempo totale per il numero di vettori

Si provi il codice sia compilato in modo Debug (quindi passando l'opzione `-DCMAKE_BUILD_TYPE=Debug` a CMake), sia in modo Release (quindi `-DCMAKE_BUILD_TYPE=Release`).

## 2 Modalità di svolgimento e consegna

Dev'essere consegnato quanto richiesto più sopra, includendo un `CMakeLists.txt` che automatizzi la compilazione dei programmi. I file devono essere posizionati in una cartella chiamata `esercitazione5` nel proprio repository.

La consegna delle esercitazioni è obbligatoria e da effettuarsi entro i termini prescritti. Il tempo assegnato per lo svolgimento di questa esercitazione è di **due settimane**. Termine ultimo per la consegna di questa esercitazione sono le ore 23.59 di Mercoledì 06/05/2026: fa fede il timestamp del commit taggato `consegna_es5` sul proprio repository. Eventuali giustificazioni vanno presentate a `matteo.cicuttin@polito.it` entro il termine ultimo previsto per la consegna dell'esercitazione.

Si ricordi che l'uso di strumenti di intelligenza artificiale è scoraggiato ed in ogni caso dev'essere dichiarato esplicitamente all'interno dei sorgenti consegnati.

All'esame potrebbe essere richiesto di mostrare il funzionamento del codice, pertanto ci si assicuri che il codice nel proprio repository sia funzionante.

## 3 Pseudocodice degli algoritmi

---

**Algorithm 1** MERGE-SORT

---

```
1: procedure MERGE-SORT( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \lfloor (p + r) / 2 \rfloor$ 
4:     MERGE-SORT( $A, p, q$ )
5:     MERGE-SORT( $A, q + 1, r$ )
6:     MERGE( $A, p, q, r$ )
7:   end if
8: end procedure
```

---

---

**Algorithm 2** MERGE

---

```
1: procedure MERGE( $A, p, q, r$ )
2:    $n_1 \leftarrow q - p + 1$ 
3:    $n_2 \leftarrow r - q$ 
4:   create arrays  $L[1 \dots n_1 + 1]$  and  $R[1 \dots n_2 + 1]$ 
5:   for  $i \leftarrow 1$  to  $n_1$  do
6:      $L[i] \leftarrow A[p + i - 1]$ 
7:   end for
8:   for  $j \leftarrow 1$  to  $n_2$  do
9:      $R[j] \leftarrow A[q + j]$ 
10:  end for
11:   $L[n_1 + 1] \leftarrow \infty$ 
12:   $R[n_2 + 1] \leftarrow \infty$ 
13:   $i \leftarrow 1$ 
14:   $j \leftarrow 1$ 
15:  for  $k \leftarrow p$  to  $r$  do
16:    if  $L[i] \leq R[j]$  then
17:       $A[k] \leftarrow L[i]$ 
18:       $i \leftarrow i + 1$ 
19:    else
20:       $A[k] \leftarrow R[j]$ 
21:       $j \leftarrow j + 1$ 
22:    end if
23:  end for
24: end procedure
```

---

---

**Algorithm 3** QUICKSORT

---

```
1: procedure QUICKSORT( $A, p, r$ )
2:   if  $p < r$  then
3:      $q \leftarrow \text{PARTITION}(A, p, r)$ 
4:     QUICKSORT( $A, p, q - 1$ )
5:     QUICKSORT( $A, q + 1, r$ )
6:   end if
7: end procedure
```

---

---

**Algorithm 4** PARTITION

---

```
1: procedure PARTITION( $A, p, r$ )  
2:    $x \leftarrow A[r]$   
3:    $i \leftarrow p - 1$   
4:   for  $j \leftarrow p$  to  $r - 1$  do  
5:     if  $A[j] \leq x$  then  
6:        $i \leftarrow i + 1$   
7:       exchange  $A[i]$  with  $A[j]$   
8:     end if  
9:   end for  
10:  exchange  $A[i + 1]$  with  $A[r]$   
11:  return  $i + 1$   
12: end procedure
```

---